60 分钟，从零做一个 Agent

用一个能运行、看得见过程的小项目，理解截图里的 9 个术语。

推荐项目：Agent 术语学习助手
输入“用生活例子解释 RAG，再问我一道题”，助手会查本地知识卡、根据证据解释，并记住你的学习偏好。
你还能展开查看它选了什么工具、传了什么参数、拿到了什么结果。

一小时后，拿到什么

一个本地 Streamlit 聊天页：左边对话，右边或下方展示 plan、工具调用记录、检索片段和来源。知识来自 notes/ 中的 Markdown；偏好写入 memory.json。

一条真正的工具闭环：模型选择工具 → Python 执行函数 → 工具结果回传模型 → 模型给出回答。
LangGraph 管理状态和循环。

时间承诺与前提

目标总时长 60 分钟，包含阅读、搭建、验证与复盘。默认你会复制命令、保存文件，并使用 Codex 等编程助手；不要求你已经理解 Agent。Python 3.11 或 3.12、可联网环境、可用的模型 API 凭据是顺利完成的前提。

第 8-16 分钟也留给安装与 API 冒烟检查；若你还需注册、充值或排查网络，60 分钟可能不够。没有可用 API 时，可运行 mock 模式观察流程，但它不算完成真实 Agent 验收。

覆盖程度，先说清楚

动手验证 8 项：RAG、Agent、Tool Calling、Function Calling、Memory、Planner、Executor、LangGraph。
理解与对照 1 项：MCP；核心版不实现 MCP 通信。它是后续替换工具连接方式的扩展，不是运行 Agent 的必需条件。

本轮只做：3 个工具、3 个图节点、小型文本知识库。
把登录、联网搜索、PDF 上传、向量数据库、多 Agent、部署和复杂 UI 留到下一轮。

9 个名词，各自负责什么

先用这张表建立位置感；实现时再回来看代码中的对应物。

术语	一句话理解	本项目中的可见证据
RAG [1]	检索增强生成：先找外部资料，再让模型依据资料回答。	search_notes 返回片段；回答带文件名和段落编号。
Agent [2]	围绕目标，模型能选择行动，并根据执行结果继续决定。	模型可选择检索、保存偏好或直接回答；不是每题都固定查库。
Tool Calling [3]	模型以结构化请求使用外部能力；运行时负责执行。	日志显示工具名、参数、结果和调用 ID。
Function Calling [3]	以函数名和参数表达工具调用的常见形式。两词常混用。	模型请求 search_notes(query=...)，Python 函数实际运行。
Memory [4]	把可用信息保留，并在之后主动读取、影响回答。	当前线程保留对话；新会话读取 memory.json 中的偏好。
Planner	提出接下来怎么做。是一种职责，可与决策模型共用。	显示 1-3 条简短行动计划；不展示隐藏思维链。
Executor	执行已选择的行动，拿到真实结果。也是一种职责。	校验参数、调用白名单函数、把结果封装成 ToolMessage。
LangGraph [2,4]	用节点、边和共享状态组织执行流程的框架。	planner、decide、tools 节点，以及工具执行后的回边。
MCP [5]	Model Context Protocol：应用连接工具和上下文的标准协议。	扩展时用客户端发现并调用服务端工具，而非直接调用本地函数。

避免混淆：RAG 是知识使用方式；Memory 是信息保留与复用；LangGraph 是编排框架；MCP 是连接协议。它们可以组合，但不是同一种东西。

Planner 与 Executor 不必分别是两个 Agent，也不必使用两个模型。关键词检索也能构成 RAG；Embedding 和向量数据库不是必要条件。资料编号见第 8 页。

把术语放进同一次请求

例子：“用做饭类比解释 RAG；记住我喜欢生活例子。”

最小 LangGraph：三个节点，一个循环

START → planner → decide
decide → 有 tool_calls → tools → decide
decide → 没有 tool_calls → END

planner：生成简短行动计划，写入 plan。
decide：读取消息、计划与偏好，由模型选工具或给最终回答。
tools：执行白名单函数，将真实结果加入消息，再回到 decide。

步骤	你应该看到的东西
1. 计划	“检索 RAG 定义；保存学习偏好；给出类比与问题。”计划是可修改的行动摘要。
2. 选择行动	模型发出 search_notes({"query":"RAG 检索 增强 生成"})。这一步体现 Function / Tool Calling。
3. 执行	Executor 在本地 Markdown 中检索，返回 rag.md#p1 等证据；也可执行 save_preference。
4. 再决定	工具结果与 call ID 回传模型。模型再决定继续调用还是结束，而非写一句“已查询”就算完成。
5. 生成	依据检索片段解释“先翻菜谱再做菜”，引用来源并出题。这段检索加生成过程就是 RAG。

状态与工具：保持小而明确

State：messages（用 add_messages reducer）、plan、tool_rounds；日志独立记录。
search_notes(query)：返回最多 3 个片段，每个带 source、chunk_id、text。
save_preference(key, value)：仅允许 example_style、answer_length 两个键。
get_preferences()：读取偏好；每次新请求也由代码预加载到模型上下文。

短期记忆用缓存于 Streamlit session_state 的 InMemorySaver 和稳定 thread_id；它不跨进程重启。长期偏好用本地 JSON 文件，跨会话读取。这是教学版自定义存储，未实现 LangGraph Store。[4]

60 分钟时间表

按闹钟推进；第 45 分钟后冻结新功能，保留验收时间。

时间	做什么	阶段产物
00-08	读第 1-3 页。用自己的话区分 RAG、Agent、MCP。	知道每个词在项目里的位置。
08-16	新建独立文件夹；检查 Python、依赖、模型及工具调用能力。	一次真实 tool call 和对应工具结果。
16-34	把第 5 页任务书交给编程助手；生成核心项目并运行。	能检索、有来源、能看执行日志。
34-45	按第 6 页补充知识卡、偏好与边界行为；修复阻塞问题。	重启后可读取学习偏好。
45-55	逐项做第 7 页的 5 条验收，保留结果。	不是只看回答“像不像”。
55-60	做第 8 页复盘，并读 MCP 对照。	能指出 9 个术语各自对应什么。

Windows：在新文件夹中执行

```
mkdir agent-terms-lab
cd agent-terms-lab
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install streamlit langgraph langchain-openai python-dotenv
.\.venv\Scripts\python.exe -m streamlit run app.py
```

最后一行等 app.py 生成后再执行。已有 Python 3.11 时把命令中的 3.12 改为 3.11。使用虚拟环境解释器可避免 PowerShell 激活脚本权限问题。

模型配置与快速止损

在 .env 配置 LLM_API_KEY、LLM_MODEL，以及按需设置 LLM_BASE_URL；由代码显式传给 ChatOpenAI。模型与端点必须支持兼容的聊天及 tool calling 接口，不能只验证普通聊天成功。API 调用可能收费，选你已有可用额度的轻量模型。

若搭建落后超过 5 分钟：先把 Streamlit 改为命令行，保留 LangGraph、检索和工具日志。API 或安装故障持续超过 8 分钟：用 mock 跑通工程并标注未完成项，后续再补真实模型。

复制给编程助手

主任务书：先跑通，再讲解

将下框全文发送给 Codex；本次方案交付 PDF，不会提前替你创建应用项目。

请在独立目录 agent-terms-lab 从零实现“Agent 术语学习助手”。我只有 60 分钟学习与搭建时间。用 Python 3.11/3.12、Streamlit、LangGraph、langchain-openai、python-dotenv。直接生成文件并运行必要检查，先保证最小闭环。

1. 文件：app.py、agent.py、tools.py、notes/*.md、.env.example、.gitignore、requirements.txt、README.md。实际密钥只在 .env；忽略 .env、memory.json、.venv。不要修改目录外文件。

2. 模型配置：从 .env 读取 LLM_API_KEY、LLM_MODEL、可选 LLM_BASE_URL，并显式传给 ChatOpenAI。先验证真实工具调用；不要假设任意兼容端点都支持 tools。缺凭据时给出清晰提示；mock 模式必须醒目标注。

3. 用 StateGraph 自建 planner、decide、tools 三个节点。State 包含 messages（add_messages）、plan、tool_rounds。planner 生成最多 3 条行动摘要；decide 用 bind_tools 选工具或结束；tools 执行后以匹配 call ID 的 ToolMessage 回到 decide。多项 tool_calls 要逐项执行并回传，避免丢失消息。

4. 工具：search_notes(query)、save_preference(key,value)、get_preferences()。只读 notes 内 Markdown。按空行切段，用中英双语术语别名与关键词打分，返回 top 3 正分片段及文件名、段落 ID。无命中返回空列表，禁止用全部文档充当检索结果。

5. 依据片段回答知识题，引用实际返回的 source#chunk_id；无证据说明“笔记里暂时找不到”。不能假造来源。随聊可直接回答；不得强制每轮都查资料。

6. 会话内用 InMemorySaver 和稳定 thread_id；graph/checkpointer 缓存于 session_state。新会话使用新的 ID。偏好仅允许 example_style、answer_length 两键，写入 memory.json；每次请求主动读取并注入上下文。

7. 页面展示回答与可展开日志：plan、节点、工具名、参数、真实结果、来源、耗时。不要显示密钥或隐藏思维链。每轮最多 3 个工具批次；到上限停止工具调用，基于已有结果输出并说明限制；模型请求设超时，错误可读。

8. 内置第 6 页知识卡内容，并增加一句不可能凭预训练知道的实验事实用于验收。只实现本地函数工具，MCP 仅在 README 解释扩展位置。不要增加登录、向量库、联网搜索或多 Agent。生成后给运行命令，并用一条真实日志说明各术语对应哪些代码。

交付要求：README 写出配置、启动方法、已安装并验证的版本；成功后再导出锁定依赖。模型与 SDK 细节按当前官方文档实现，不要混用不同消息协议。

把输入压小，把效果看清楚

给助手下面这批种子资料，再用增量提示修复缺口。

可直接放进 notes/ 的三张卡

rag.md
RAG / 检索增强生成：按用户问题检索外部资料，把命中的片段提供给模型，再生成有依据的回答。检索可以用关键词，不一定需要向量数据库。
本项目实验约定：蓝鲸计划的学习暗号是“橘子灯塔 731”。这是虚构的本地实验事实，不是行业知识。

agent_tools.md
Agent / 智能体：模型围绕目标选择工具或回答，并根据工具结果继续行动。固定顺序调用全部函数更接近工作流。
Tool Calling / 工具调用：模型提出结构化请求，执行环境调用工具并返回结果。Function Calling / 函数调用是常见表达形式，两词在不少 API 中近似使用。
Planner / 规划器提出行动计划；Executor / 执行器执行行动。它们是职责，不要求拆成多个 Agent。

memory_graph_mcp.md
Memory / 记忆：保存并读取历史或偏好，让后续回答发生变化；仅把信息写入文件还不够。
LangGraph：用图节点、边和共享状态组织应用；条件边决定执行工具还是结束，回边支持循环。
MCP / 模型上下文协议：应用中的客户端与服务端交互，发现工具并调用。它也支持资源和提示词等上下文能力；它不规定 Agent 怎样规划。

增量提示：只在核心版跑通后使用

“请核对：问‘蓝鲸计划的学习暗号是什么’，是否真实执行 search_notes，并引用 rag.md 的实际段落编号？问‘我的会员积分余额是多少’，是否报告无证据？若不满足，只修检索、提示词和日志，不重构 UI。”

“请完成偏好实验：我说‘记住我喜欢生活类比’，确认写入 example_style；新开会话再问 RAG，展示读取到的偏好。重启程序后重复。仅文件存在不算通过。”

遇到错误时，给编程助手这段话

“当前失败步骤是【启动 / API / 工具调用 / 检索 / 记忆】。报错与复现输入如下：【粘贴完整错误，隐去密钥】。请先定位原因，再做最小修复，复跑失败输入并贴证据。不要更换整套技术栈。”

用 5 个实验证明它真的工作

记录真实结果；以下是验收预期，不代表本 PDF 已附带或测试了应用。

输入 / 操作	通过标准	理解的概念
① “蓝鲸计划的学习暗号是什么？”	出现 search_notes 的真实日志；答案为“橘子灯塔 731”，引用命中段落。临时修改暗号后答案随之改变。	RAG、工具调用
② “你好”再问“RAG 是什么？”	寒暄可以直接回答；术语问题选检索。能看到调用参数、执行结果和模型最终回答。	Agent、Function Calling
③ “记住我喜欢生活类比”	保存成功；新会话及重启后读取偏好，再问术语时应用生活类比。旧对话不自动进入新会话。	Memory
④ “我的会员积分余额是多少？”	本地无命中；回答找不到证据，不编造数字或来源；日志真实显示空结果。	RAG 的边界
⑤ 展开日志，并模拟工具报错	指出 planner、decide、tools 的顺序及回边；错误作为工具结果处理；超出调用上限可停止。	Planner、Executor、LangGraph

MCP 在哪里接入？[5]

核心版：decide → 本地 Executor → search_notes Python 函数。
MCP 版：decide → Executor / 工具适配层 → MCP Client → MCP Server → search_notes。

客户端先用 tools/list 发现工具及参数说明，再用 tools/call 请求执行。服务端提供能力；模型仍负责选择行动。MCP 接通不等于自动具备规划或记忆。

本小时的达标线：能画出上述两条链路，并解释“换了连接方式，检索与生成逻辑可以不变”。未运行客户端与服务端，不应称为“已完成 MCP 接入”。

下一轮扩展任务：把 search_notes 包成只读 MCP Server，用官方 Python SDK 与客户端适配层连接；真实发现工具并调用一次，核对返回来源，再接回 decide。控制在单个本地服务；另留 20-40 分钟，这是估计，未计入本小时。

合上 PDF，试着讲清楚

不背定义；用你刚看到的输入、状态、调用与结果解释。

5 分钟复盘：每题一句话

1. 删除 search_notes 后，哪部分证据链消失？为什么只是回答正确不足以证明 RAG?
2. 模型输出工具名和参数后，究竟是谁执行 Python 函数?
3. 如果 memory.json 有偏好，但之后从不读取，它算有效记忆吗?
4. planner 与 tools 是职责还是必须拆分的两个 Agent?
5. LangGraph 的哪条边让工具结果重新进入模型?
6. 换成本地 MCP Server 后，哪段连接变了，哪段回答逻辑可以保留?

自检答案：检索证据消失，模型可能靠预训练猜对；执行环境负责跑函数；记忆必须被读取并影响后续行为；规划与执行是职责；tools → decide 是回边；MCP 改的是工具接入与发现机制。

你现在的完成标准

核心 5 项验收通过，且能在代码或日志中指出 8 个实践概念，并解释 MCP 的接入位置。若用了 mock，记录“工程流程已跑通，真实模型决策待验证”。这是入门原型，不等于熟练掌握全部技术。

官方资料：需要时再读对应小节

资料于 2026-10-22 核对。以下为概念依据；方案中的时长、文件拆分与验收规则是本方案的教学设计。

[1] Retrieval：理解检索与生成的关系
https://docs.langchain.com/oss/python/deepagents/retrieval

[2] Workflows and agents：固定流程与动态决策
https://docs.langchain.com/oss/python/langgraph/workflows-agents

[3] Function calling：工具请求、执行与结果回传
https://developers.openai.com/api/docs/guides/function-calling

[4] Persistence：线程检查点与跨线程存储
https://docs.langchain.com/oss/python/langgraph/persistence

[5] MCP Architecture：Host / Client / Server 与工具交互
https://modelcontextprotocol.io/docs/learn/architecture

资料阅读顺序：先做项目，卡在哪个概念就读哪一篇。第一小时不要求通读完整文档。